INDIAN PEDIATRICS
AI ASSISTANT

STEP 3

Data Processing

A beginner-to-practitioner guide to page-level PDF extraction, conservative cleaning, provenance, visual exceptions, and a repeatable data-quality gate.

Teaching manual installment 1.0 | October 2026

Project: Reusable medical-journal literature assistant
POC corpus: Indian Pediatrics, January 2025, Volume 62, Issue 1
Audience: AI students and developers learning to plan and build a grounded literature assistant

How to use this installment
These are the first chapters of a cumulative teaching manual. They explain decisions made so far and provide a reproducible way to understand them. The full 300+ page reference is intended to grow as later roadmap phases are completed.

Document control and alignment

This chapter follows **Phase 3 - Data Processing** in the project roadmap and the four numbered activities in the Implementation Guide: **14. PDF/HTML extraction, 15. Cleaning, 16. Metadata attachment, and 17. Quality inspection.** Only PDFs were supplied for the selected January 2025 Indian Pediatrics issue, so this implementation processes PDF files. There were no acquired HTML article files to process.

Step 3 outcome
17 acquired article PDFs were extracted page by page, producing 77 page records. Two Step 2 administrative/mixed files were excluded by policy. Source hashes confirm the 17 source PDFs were not changed. Quality review found one visual limitation: the systematic review's printed page 59 is a PRISMA flow-chart image and its counts are not searchable extracted text.

Contents at a glance

- 1. Purpose, inputs, outputs, and boundary
- 2. Activity 14 - extract PDF/HTML text
- 3. Page provenance and article structure
- 4. Activity 15 - clean conservatively
- 5. Activity 16 - attach metadata
- 6. Activity 17 - inspect quality
- 7. Processing architecture and code walkthrough
- 8. Recorded results and known exception
- 9. Reproduction, troubleshooting, and handoff

1. Purpose, inputs, outputs, and boundary

Step 3 converts acquired documents into traceable text artefacts suitable for later chunking and retrieval. It must retain enough provenance to answer: which journal article did this text come from, which local source file, which physical PDF page, and which printed page if that mapping is known?

Extraction does not mean interpretation. Cleaning removes a small class of mechanical artefacts while preserving the author's words, citations, headings, and page order. It must not summarize, correct medical claims, infer missing metadata, or silently reconstruct tables and figures.

1.1 Inputs

- Step 2 `article_metadata.csv` and `corpus_file_manifest.csv`.
- The 19 source PDFs under `data/raw/2025_Jan/`.
- Explicit include/exclude decisions from Step 2.
- A pinned PDF parser dependency and a stable local project environment.

1.2 Outputs

Output	Purpose
data/processed/extracted/.jsonl	One JSON record per PDF page, retaining raw and cleaned text, article metadata, physical and printed page references, role, image count and source hash.
data/processed/cleaned/.txt	Readable article text with visible page boundary markers for inspection and future chunking.
data/processed/quality/processing_manifest.csv	One row per Step 2 file with processing status, source checksum, parser version, page totals and output paths.
data/processed/quality/page_quality.csv	One row per processed page with character counts, image count, page role and review status.
data/processed/quality/visual_review_log.csv	Human-readable decision for a page that has meaningful graphic content but little selectable text.
data/processed/step3_quality_report.md	Summary of checks, results, limitations and commands to reproduce the run.

1.3 Out of scope

- OCR or manual transcription of figure/table contents.
- Chunking, embeddings, vector database, retrieval or LLM calls.
- Clinical interpretation or correction of article content.
- Fetching HTML or website content when no HTML corpus has been acquired.
- Permission/licence determination.

2. Activity 14 - extract PDF/HTML text

PDFs store text and layout differently. Some contain searchable characters, some are images, and many mix normal text with tables and figures. Extraction is therefore a transformation with a measurable quality risk, not a guaranteed conversion.

2.1 Why process page by page?

If an entire PDF becomes one text string, a missing or scrambled page is hard to locate. Page-level records allow every result to be traced to a source page, let a reviewer compare a page with its rendered appearance, and give later chunking a stable citation anchor.

2.2 What the current processor does

- Opens each PDF read-only with pypdf 6.10.0.
- Enumerates physical pages starting at 1.
- Extracts selectable text separately for each page.
- Records physical page number, article ID, source filename, SHA-256 checksum and parser version.
- Maps physical pages to printed article pages when page-range metadata makes that mapping direct.
- Labels extra pages of the 56.pdf systematic review as unpaginated/supplement pages rather than dropping them.
- Writes both original extracted text and conservatively cleaned text into page-level JSONL.
- Writes a page-marked plain-text review copy for people and future chunking.

2.3 Physical page and printed page are different

A physical page is the position in the supplied PDF. A printed page is the page number visible in the journal. Citations and provenance need the printed page where known; debugging extraction needs the physical PDF page. They must remain separate. For example, 56.pdf has 13 physical pages, while the article's printed pages run 56-62 and later PDF pages are supplemental material without journal pagination.

2.4 HTML case

The roadmap allows PDF or HTML sources. This corpus contains PDFs only, so the run uses no web crawler and does not fetch article pages. For a later HTML corpus, add a source adapter that preserves title, article sections, headings, tables, canonical URL and source timestamps; keep the same article/page-or-section provenance contract. Do not run a scraper just because the framework mentions HTML.

3. Page provenance and article structure

A reliable text passage must retain its parent article identity. Later chunk IDs should link back to an article record and, where possible, a page and section. Step 3 lays that foundation before chunking exists.

3.1 Page record fields

Field	Reason
article_id, title, journal, section, content_role	Connect page text with article catalogue and evidence-role policy.
issue_id, year, volume, issue_number	Support issue/time filtering and citations.
source_filename, source_relative_path, source_sha256	Identify the exact unchanged source and detect accidental replacement.
physical_page, printed_page, page_role	Keep PDF location distinct from journal citation page and supplementary material.
raw_text, cleaned_text	Allow audit of each normalization and future reprocessing.
embedded_image_count, character_count, quality_status	Make visual extraction gaps visible instead of treating sparse text as complete.
extractor name/version in processing manifest	Make processing reproducible and comparable after parser upgrades.

3.2 Structure preservation policy

This implementation preserves page boundaries and extracted line breaks. It does not attempt to detect every heading, paragraph, table cell or figure caption as a separate data object. Pypdf's text output can follow a PDF's internal reading order, which may differ from visual reading order in a two-column page. Keep the page JSONL as the audit layer. More advanced layout-aware extraction can be evaluated later on difficult pages rather than applied blindly to all pages.

4. Activity 15 - clean conservatively

Cleaning should improve text usability without changing meaning. Every clean operation should be deterministic and inspectable. When in doubt, preserve the source text and report the issue instead of guessing.

4.1 Normalizations used

- Unicode NFC normalization to combine canonically equivalent character sequences.
- Normalize Windows, carriage-return and line-feed variants to newline characters.
- Remove soft hyphen and zero-width formatting artifacts that may be inserted by text layout.
- Expand common presentation ligature code points such as fi/fl into ordinary letter sequences.
- Convert nonbreaking spaces and tabs to regular spaces; collapse repeated horizontal spaces.

- Trim line edges and collapse runs of three or more newlines while retaining paragraph breaks and page separation.
- Retain punctuation, minus signs, decimal values, units, citations, references and original line order.

4.2 Deliberately not done

- No automatic spelling correction, paraphrasing, summarization or medical terminology rewriting.
- No automatic joining of hyphenated line endings: some hyphens are genuine words, and a false merge is hard to detect.
- No removal of headers, footers, references or repeated content at this stage; later filtering needs its own checks.
- No conversion of tables or figures into guessed prose.
- No replacement of an unknown glyph with a guessed character.

Raw plus clean is an audit pair
The JSONL keeps the parser output and normalized text side by side. If a later model answer looks wrong, the team can determine whether the defect came from the source PDF, extraction, normalization, chunking, retrieval or generation.

5. Activity 16 - attach metadata

Metadata is attached at the page record level from the Step 2 article catalogue and file manifest. This avoids asking the language model to guess authors, date, issue, category or source link from prose. The processing manifest remains separate from the article catalogue so each stage can report its own status.

5.1 Metadata attachment rules

- Use the stable article_id as the join key between the catalogue, manifest, page text and future chunks.
- Carry article title, section and content_role into each page record so downstream filters can act locally.
- Copy publication fields from validated catalogue records; do not infer missing author or DOI values.
- Keep an empty printed page value when it cannot be mapped confidently.
- Keep supplements explicitly labelled and linked to the article while retaining their physical page number.
- Verify the first extracted page against the catalogue title; a mismatch is a data-integrity warning.

5.2 Why no author matching by PDF text?

PDF title pages often contain author names and affiliations, but extracting and normalizing them is a separate metadata validation task. Automated parsing can merge affiliations into names or split compound names. The present pipeline attaches verified Step 2 values and leaves unknown fields blank. Author-search completeness is a later enrichment task, not a reason to invent data.

6. Activity 17 - inspect quality

A correct program exit is not enough. The quality check compares file identity, page counts, article IDs, page order, text presence, normalization output, and a text-to-catalogue title match. It also creates explicit exceptions when a page has little selectable text.

6.1 Automated quality checks

Check	Defect it can detect
Manifest coverage	A Step 2 file was omitted or an unlisted file entered processing.
Source SHA-256 equality	A raw source changed after processing or outputs point at a replacement file.
PDF page count vs page-record count	A page was skipped, duplicated or miscounted.
Continuous physical page sequence	A provenance gap in page records.
Article ID and source-hash consistency	Text is joined to the wrong article or source.
Nonblank page text	A missing text layer or unusable page.
Title token overlap	Possible mismatch between issue metadata and local PDF.
Replacement-character scan	Unicode decode failures that could corrupt names or medical terms.
Supplement role coverage	Extra pages were not silently discarded.
Visual review log	Sparse/graphic pages have an explicit, reviewable disposition.

6.2 Human visual review

Automated counts cannot tell whether a figure is meaningful or a table was reordered. Render flagged and representative pages and compare them with extracted text. The review should record the PDF file, physical page, printed page, what was observed, what the text layer missed, and the decision. Do not write a human transcription as if it were publisher text; label its source and require verification.

6.3 Thresholds are review triggers, not truth

The current 200-character threshold flags a page for review; it does not automatically mean extraction failed. A page can be a diagram, a full-page table, or mostly white space. Conversely, a page with thousands of characters can still have columns in the wrong order. Thresholds guide inspection; visual comparison and provenance checks provide context.

7. Processing architecture and code walkthrough

The workflow is intentionally two commands: the processor creates deterministic outputs; the checker audits them and writes a report. Separating the two makes it possible to rerun validation without recomputing all text, and to inspect changes after parser updates.

Processor: `scripts/process_step3.py`
Checker/report: `scripts/check_step3.py`
Library: pypdf 6.10.0, pinned in `requirements.txt` and `pyproject.toml`

7.1 Processing sequence

- Read both Step 2 CSVs and map article_id to catalogue metadata.
- Iterate the source manifest in stable order; hash the raw file before reading.
- Skip only explicit administrative duplicate/news roles or files without article identity.
- Open each PDF without modifying it; reject encrypted/unreadable inputs visibly.
- Extract each page; normalize text; calculate page role, character counts and image counts.
- Write JSONL pages and page-marked text using article IDs as filenames.
- Write one processing-manifest row for every input file, including excluded files.
- Run checker, reconcile failures and review flags, and retain the report.

7.2 Commands

From a PowerShell window opened in the Project root:

```
.\.venv\Scripts\Activate.ps1
python scripts\process_step3.py
python scripts\check_step3.py
```

Use the same pinned dependency version when regenerating the corpus. If the extractor version changes, retain the old manifest/report or commit a new versioned run so differences can be audited.

8. Recorded results and known exception

The current checked run reports **12 PASS, 0 FAIL, 1 documented visual limitation**. It is ready to hand off to Phase 4 only with that limitation carried forward.

8.1 Quantitative summary

Measure	Result
Manifest entries	19 physical PDFs listed
Article PDFs processed	17
Administrative/mixed PDFs excluded	2 (duplicate commentary ending/access notice; News in Brief)
Pages represented	77
Page text records	77 ordered page-level JSONL records
Unpaginated/supplement pages retained	6 pages from 56.pdf
Blank extracted pages	0
Unicode replacement characters	0
Title-to-first-page token overlap	17 of 17 exceeded the 80% check threshold
Raw source checksum changes	0

8.2 Documented visual limitation

The systematic review PDF (`56.pdf`), physical page 4 / printed page 59, contains Figure 1, a PRISMA flow chart. Pypdf extracts only the page header and figure caption (90 cleaned characters); one embedded image is detected. Visual review confirms the page is not blank and contains screening-flow counts. Those counts are not part of searchable text and must not be cited as if they were retrieved. The original page remains preserved and the limitation is listed in `visual_review_log.csv` and the quality report.

> **Phase 4 guardrail**
> The article's surrounding searchable prose may be used subject to the normal evidence checks. Do not answer a question that depends on the diagram's exact counts from this text index until a separately labelled OCR or manual transcription has been checked against the source figure.

8.3 Quality interpretation

All files were structurally readable, all 77 physical pages have ordered records, the source hashes remain unchanged, and the article titles agree with the first-page text. That is a strong extraction baseline. It is not proof that every table cell, diagram label, formula, or two-column reading order is accurate. Review the source page when the answer depends on fine-grained layout content.

9. Reproduction, troubleshooting, and handoff

A fresh developer should be able to regenerate the derived files from the source PDFs and compare the resulting quality report. Raw inputs remain read-only; delete or replace only derived outputs when intentionally rerunning the processor.

Symptom	Likely cause	Action
Import error for pypdf	Dependency not installed in the active virtual environment.	Activate `.venv`; install from the project requirements file.
PDF count differs from manifest	A file was added/removed after Step 2 inventory.	Return to corpus inventory, update manifest deliberately, then rerun.
Encrypted or malformed PDF	Input cannot be read by parser.	Do not silently skip; record failure and obtain a usable authorized copy.
A page has very few characters	Graphic/table page, scan, or extraction defect.	Render page, inspect it, log disposition; OCR or transcribe only with verification.
Page/heading order looks wrong	PDF text order differs from visual columns.	Retain raw PDF and evaluate layout-aware extraction on that page; compare outputs before adopting.
Clean text differs unexpectedly	Normalization may have changed an artifact.	Compare raw_text and cleaned_text in JSONL and adjust only deterministic cleaning rules.

9.1 Phase 4 handoff checklist

- Run the Step 3 checker and retain its exact output/report.
- Ensure all required article outputs exist and source hashes match.
- Carry the PRISMA figure exception into the index policy and answer safeguards.
- Decide which content roles are eligible for each query type; nonclinical education content should not support clinical advice.
- Confirm supplement page treatment for 56.pdf before chunking its appendices.
- Only then proceed to chunking while retaining article ID, section and page provenance on every chunk.

References and glossary

- Ver 1 - Complete Implementation Guide for POC.pdf, Phase 3 Data Processing.
- Ver 1 - Master Guide for POC.pdf, sections on extraction, data quality and chunk provenance.
- Project scripts/process_step3.py, scripts/check_step3.py, data/processed/step3_quality_report.md, page_quality.csv and visual_review_log.csv.

Glossary

Term	Plain-language meaning
Physical page	One-based position in the PDF file.
Printed page	Journal page number shown on the document, if confidently mapped.
JSONL	A text file containing one JSON object per line; here, one page record per line.
SHA-256	A cryptographic checksum used to identify whether source bytes changed.
OCR	Optical character recognition: machine reading of text from page images.
Page provenance	Metadata linking extracted text to source file, article and page.
Visual limitation	Meaningful page content is present in the PDF but is not available as searchable extracted text.